

Proyecto 1 Arquitectura de computadores

Alejandro Henao Cordero
Escuela de Ingeniería Electrónica
Tecnológico de Costa Rica
Cartago, Costa Rica
ahenao@estudiantec.cr

María Paulina Camacho Herrera
Escuela de Ingeniería Electrónica
Tecnológico de Costa Rica
Cartago, Costa Rica
paulinacamacho@estudiantec.cr

I. INTRODUCCIÓN

En este trabajo se aborda el diseño, la implementación y la evaluación comparativa entre una solución escalar y una solución vectorizada desarrollada en lenguaje ensamblador x86-64 utilizando NASM. Se desea construir un programa que, dado un arreglo grande de números en punto flotante de decisión simple, calcule sus estadísticos descriptivos y produzca una versión normalizada del arreglo. Se calculan valores como la suma total de los datos, media, varianza, desviación estándar, valor mínimo y valor máximo. Posteriormente, se genera una versión normalizada mediante el cálculo de la puntuación z para cada elemento del arreglo.

El objetivo principal de este proyecto consiste en desarrollar e implementar un kernel escalar y un kernel vectorial mediante el conjunto de instrucciones AVX2, capaz de procesar bloques de 8 elementos de forma simultánea o de manera individual dependiendo de la versión de kernel que se use. Asimismo, se busca aplicar restricciones de alineación de memoria a 32 bytes y diseñar algoritmos de gestión de remanentes que garanticen precisión de los cálculos. Finalmente, se busca verificar la exactitud numérica de ambas soluciones frente a casos de prueba controlados bajo una tolerancia relativa estricta de 1×10^{-4} , inspeccionar el comportamiento de bajo nivel mediante el depurador GDB y evaluar cuantitativamente el tiempo de ejecución y la ganancia de velocidad (speedup) lograda a través de diferentes cantidades de datos de entrada.

El documento se encuentra organizado en varias secciones que abarcan la fundamentación teórica de las arquitecturas vectoriales, las especificaciones del entorno de pruebas y la descripción del código desarrollado. Posteriormente, se presentan los casos de prueba de verificación, el análisis de rendimiento obtenido y las sesiones de GDB. Finalmente, se realizan conclusiones y se complementa el informe con un anexo que incluye la documentación gráfica de la arquitectura, diagramas de flujo, tablas de registros y el esquema de memoria.

II. INVESTIGACIÓN PREVIA

En la arquitectura de computadoras, el procesamiento se encarga de seguir el modelo SISD (Single Instruction Single Data), también conocido como el modelo escalar. En este modelo, la ALU (Unidad Aritmética Lógica) se encarga de procesar un solo dato por ciclo. Para optimizar problemas con alto nivel de paralelismo de datos (ya sea procesamiento de señales o cálculos estadísticos) se introdujo el modelo SIMD (Single Instruction Multiple Data), que permite que una sola instrucción opere simultáneamente sobre varios datos que han sido empaquetados dentro de un registro más ancho, reduciendo de esta manera, la cantidad de instrucciones ejecutadas por el procesador. [4, p168] A diferencia de los registros de propósito general, x86 utiliza bancos de registros dedicados.

SSE/SSE2 (1999 - 2001): Introdujeron los primeros registros vectoriales dedicados de 128 bits, nombrados desde XMM0 hasta XMM15.

AVX/AVX2 (2011 - 2013): Expandieron el ancho del hardware a 256 bits, introduciendo los registros desde YMM0 hasta el YMM15. AVX2 permite operar en paralelo con números enteros y con datos de tipo punto flotante.[2, p44] Se tienen dos tipos de instrucciones: las instrucciones de tipo escalar y las instrucciones de tipo vectorial.

Instrucciones escalares: Operan solo un único dato a la vez. Toman un solo número de punto flotante de 32 bits y lo procesan utilizando únicamente los 32 bits más bajos de un registro XMM de 128 bits, mientras que los 96 bits restantes del registro se ignoran. Algunos ejemplos son movss, addss, mulss. La instrucción movss mueve un solo float de la memoria al registro, addss suma dos floats individuales y mulss se encarga de multiplicarlos.

Instrucciones vectoriales SSE (movaps, Addps, mulps): Operan sobre 4 datos de manera simultánea. Para este tipo de instrucciones el sufijo ps indica Packed - Single Precision. Utilizan la totalidad de los 128 bits de los registros XMM, como un float ocupa 32 bits, caben 4 números en el registro, mientras que por su parte, la ALU ejecuta la operación matemática en los 4 carriles al mismo tiempo en un solo ciclo de reloj.

Instrucciones vectoriales AVX/AVX2: Operan sobre 8 datos de manera simultánea. Para este tipo de instrucciones el prefijo v indica el uso de la codificación VEX. Utilizan los registros YMM, que son el doble de ancho de capacidad (256 bits). Esto permite empaquetar y procesar 8 valores tipo float en paralelo. Mientras que la instrucción vmoaps mueve un bloque de 8 números, vaddps y vmulps suman o multiplican a todos al mismo tiempo.

Restricciones de memoria: alineadas vs no alineadas: El procesador necesita mover bloques de 128 o 256 bits entre la memoria RAM y los registros, la forma en como se solicita esta memoria, va a cambiar el rendimiento, se tienen dos tipos:

Movaps/vmoaps (alineadas): exigen a nivel de hardware que la dirección de memoria del arreglo sea un múltiplo exacto de 16 bytes (en el caso de SSE) o 32 bytes (para AVX). Si se intenta leer desde una dirección no alineada, el hardware genera un segmentation fault provocando el colapso del programa. [5, p391]

Movups/vmovups (no alineadas): permiten leer bloques de datos desde cualquier dirección de memoria, sin importar si es múltiplo de 16 o 32 bytes. Sin embargo, obligan al controlador de memoria a realizar más micro-operaciones internas, reduciendo de esta manera el rendimiento.

Técnicas de reducción horizontal: Las instrucciones de tipo SIMD operan de forma vertical, pero al necesitar sumar todos los elementos dentro del mismo arreglo se necesita sumar los carriles del mismo registro entre si. Hay tres formas de lograr la reducción horizontal:

vhaddps: corresponde a una instrucción dedicada a sumar horizontalmente pares adyacentes dentro del mismo registro (por ejemplo, suma el carril 0 con el 1, y el 2 con el 3)

shuffles (vshufps): reordena o "baraja" artificialmente los datos dentro del registro. Al mover los datos de lugar, se usa una suma vectorial normal (vaddps) para ir acumulando los valores.

vextractf128: toma un registro de 256 bits (YMM) y extrae su mitad superior (los 128 bits más altos) para guardarla en un registro XMM más pequeño, permitiendo de esta manera "partir a la mitad" el vector grande y sumar la mitad superior con la mitad inferior, reduciendo los datos de 8 a 4.

Reducción horizontal: Cuando se calculan estadísticos como es el caso de la sumatoria total o la media de un arreglo, no sirve tener 8 subtotaes aislados dentro de un registro YMM, se necesita un único valor final. La reducción horizontal consiste en sumar los carriles internos del registro entre sí. Esto requiere el uso de instrucciones de instrucciones especiales como vhaddps (sumas horizontales), shuffles (para reordenar datos internamente) o extraer la mitad superior de los 128 bits con vextractf128 para luego sumarla con la mitad inferior.

Alineación de memoria: Para que los buses de datos muevan bloques de 256 bits de manera eficiente, la memoria debe estar alineada. La instrucción vmoaps exige strictly que la dirección base del arreglo en memoria sea múltiplo de 32 bytes, si no lo es, el procesador generará una excepción (fault) a nivel de hardware. Aunque existen alternativas como vmovups que no exigen la alineación de memoria, pero se obtienen penalizaciones de rendimiento.

Remanente (tail loop): Si el tamaño del arreglo no es un múltiplo exacto de 8, el bucle SIMD no puede procesar los últimos elementos, por lo que se debe implementar un bucle de cierre utilizando instrucciones de tipo escalares para procesar el remanente de forma adecuada.

System V AMD64 ABI: Corresponde a la convención estándar de Linux que indica como el driver en C se comunicará con el código desarrollado en NASM. Define de forma estricta cuáles registros (RDI o RSI) transportan argumentos de la función y que registros se mantienen intactos para no corromper la ejecución del C.

Limitaciones teóricas del rendimiento: Al medir el speedup (tiempo escalar / tiempo vectorial) rara vez se alcanza el factor de aceleración teórico de 8x por dos factores. Primero la ley de Amdahl, que demuestra como el código secuencial limita la mejora máxima. Y segundo, el ancho de banda de memoria, que frena el rendimiento cuando el CPU se queda inactivo esperando que la memoria RAM suministre datos a la cache L1 o L2

III. DESCRIPCIÓN DEL ENTORNO DE PRUEBAS UTILIZADO

Para el desarrollo del proyecto se hizo uso de 2 computadoras distintas. Una laptop con procesador Intel Core i7-7700HQ (4 núcleos, 8 hilos, frecuencia máxima de 3.80 GHz) y una configuración de caché de 128 KiB L1d, 128 KiB L1i, 1 MiB L2 y 6 MiB L3, fue la máquina utilizada para realizar el benchmarks de rendimiento de los kernel escalar y vectorial implementados. Esta hizo uso de las versiones de software GCC 13.3.0, NASM 2.16.01 y GDB 17.1. La segunda computadora utilizada cuenta con un procesador AMD Ryzen 9 8945HS (8 núcleos, 16 hilos, frecuencia máxima de 5.26 GHz) y una caché de 256 KiB L1d, 256 KiB L1i, 8 MiB L2 y 16 MiB L3. Esta se utilizó para las sesiones de depuración en GDB. GCC 15.2.0, NASM 3.01 y GDB 17.1. Ambas computadoras fueron utilizadas para la escritura del código del kernel escalar y vectorial. Asimismo se verificó el soporte de ambas computadoras para el uso de instrucciones vectoriales. Esto se muestra a continuación para la computadora con el procesador AMD (también se realizó en la otra computadora):

```
alejandra-henao@alejandra-henao-R0G-Zephyrus-G14:~$ lscpu | grep -i avx
Flags: fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ht syscall nx mmxext fxsr_opt pdpe1gb rdtscp lm constant_tsc rep_good amd_l
br_v2 v2 nopl xtopology nonstop tsc cpuid extd_apicid aperfmperf rapl pni pclmulqdq monitor ssse3 fma cx16 sse4_1 sse4_2 x2apic movbe popcnt aes xsave avx f16c rdrand lahf_lm cmp_legacy svm extapic cr8_legacy abm sse4
a misalignsse 3dnowprefetch osvw ibs skinit wdt tce topoext perfctr_core perfctr_nb bpext perfctr_llc mwaitx cpuid_fault cpb cat_l3 cdp_l3 hw_pstate ssbd mba perfmon_v2 ibrs ibpb stibp ibrs_enhanced vmmcall fsgsba
e bmi1 avx2 smep bmi2 erms invpcid cqm rdt_a avx512f avx512dq rdseed adx snap avx512fma clflushopt clwb avx512cd sha_ni avx512bw avx512vl xsaveopt xsavec xgetbv1 xsaves cqm_llc cqm_occup_llc cqm_nbm total_cqm_nbm
local_user_shstk avx512_bf16 clzero irperf xsaveerptr rdpru wbnoinvd arat npt lbrv svm_lock nrip_save tsc_scale vmcb_clean flushbyasid decodeassists pausefilter pfthreshold vgif x2avic v_spec_ctrl vnni avx512vbmi u
nit_pku ospke avx512_vbmi2 gfni vaes vpclmulqdq avx512_vnni avx512_bitalg avx512_vpopcntdq rdprd overflow_recov succor snca frsm flush_l1d
alejandra-henao@alejandra-henao-R0G-Zephyrus-G14:~$
```

Figura 1: Comprobación del soporte de instrucciones vectoriales.

IV. CÓDIGO IMPLEMENTADO

A continuación se explica la implementación del kernel escalar y vectorial en ensamblador con extractos del código. Primero se realizaron los diagramas de flujo para las distintas funciones sum_array, compute_stats y normalize_array [1]. Tras validar el flujo, se procedió a escribir los algoritmos de manera ordenada.

IV-A. Kernel escalar

A continuación se muestra la imagen del algoritmo relacionado a la función sum_array versión escalar con su debida explicación:

```

31 ; -----
32 sum_array:
33     xor     eax, eax           ; eax = i = 0
34     xorps   xmm0, xmm0        ; xmm0 = acumulador = 0.0
35
36 .sum_loop:
37     cmp     eax, esi
38     jge     .sum_done
39     movss   xmm1, [rdi + rax*4]
40     addss   xmm0, xmm1
41     inc     eax
42     jmp     .sum_loop
43
44 .sum_done:
45     ret
46 ; -----
47 ; void compute_stats(const float *arr, int n,
48 ;                   float *mean, float *var, float *min, float *max)
49 ;   rdi = arr, esi = n, rdx = mean*, rcx = var*, r8 = min*, r9 = max*
50 ;

```

Figura 2: Código para la implementación de sum_array escalar.

Como se observa en la imagen, la versión sum array se inicializa en el índice i en el registro EAX y el acumulador en XMM0 mediante xorps, seguidamente el bucle .sum_loop compara i contra n (ESI) y, mientras no se alcance el límite, carga un único flotante de 32 bits con movss desde $[rdi + rax*4]$, lo suma al acumulador con addss e incrementa el índice de uno en uno.

A continuación se muestran las imágenes de los algoritmos relacionados a la función compute_stats para la versión escalar con su debida explicación:

```

91 compute_stats:
92     ;-----
93     ; Entradas de la función
94     ; Registros que almacenan los datos de entrada
95     ; Se almacenan en una pila
96     push    rbp ; guarda rbp en la pila (la función lo va usar para "arr")
97     push    rbp ; guarda rbp (se usa para "n" (cantidad de elementos de arreglo de in))
98     push    r12 ; guarda r12 (se usa para mean_ptr (puntero de la media))
99     push    r13 ; guarda r13 (se usa para var_ptr (puntero de la variancia))
100    push    r14 ; guarda r14 (se usa para min_ptr (puntero del dato minimo del arreglo))
101    push    r15 ; guarda r15 (se usa para max_ptr (puntero del dato maximo del arreglo))
102
103    ; ----- copiar los argumentos recibidos por la ABI a registros que sobreviven la llamada a sum_array -----
104    ; ----- inicio de los punteros que almacenan los datos de las pilas iniciales -----
105    ; ----- función: recibir los parámetros de la función recibidos por el profe -----
106
107    mov     rbp, rdi           ; rbp = arr (puntero al primer elemento del arreglo)
108    mov     esi, esi           ; esi = n (cantidad de elementos, 32 bits)
109    mov     r12, rcx           ; r12 = mean_ptr (donde se debe escribir la media)
110    mov     r13, rcx           ; r13 = var_ptr (donde se debe escribir la variancia)
111    mov     r14, rcx           ; r14 = min_ptr (donde se debe escribir el minimo)
112    mov     r15, rcx           ; r15 = max_ptr (donde se debe escribir el maximo)
113
114    ; ----- generación de errores -----
115    ; se guarda el valor minimo final y el valor maximo final en las direcciones de memoria apuntadas por r14
116    ; ----- caso borde: arreglo vacío o n invalido -----
117    test    esi, esi           ; test ebp, ebp si es 0, si es 0 (r14 modificar ebp)
118    jle     .compute_stats_arreglo_vacio ; si n <= 0, ir directo al bloque que llena todo con 0.0
119
120    ; ----- paso 1: media -----
121    ; ----- Reutilizando sum_array creado por el profe en el instructivo para la suma del arreglo de datos float -----
122    ; (primer ciclo) rean utilizando el sum_array
123    ; inicio de la media
124
125

```

```

126     ; primer argumento para sum_array: arr
127     mov     esi, esi
128     mov     edi, esi
129     call    sum_array
130     ; sum = suma total de arr[0..n-1]
131     ; convierte a float y lo pone en sum
132     cvtsi2ss xmm0, esi
133     divss   xmm0, xmm0
134     movss   [rdi], xmm0
135     ; *mean_ptr = media (se escribe el resultado directo en la memoria)
136     ; retorno de la media en el registro xmm0, para su posterior uso en el siguiente ciclo
137
138     ; ----- paso 2: se hace recorrido para min, max y acumulador de variancia -----
139     mov     rax, esi           ; i = 0 (inicio del segundo recorrido)
140     movss   [rdi], xmm0        ; min = arr[0] (valor inicial de referencia para comparar)
141     movss   [rdi+4], xmm0      ; max = arr[0] (valor inicial de referencia para comparar)
142     movss   [rdi+8], xmm0      ; acumulador de la variancia = 0.0
143
144     .compute_stats_loop_principal:
145     ; inicio del ciclo: procesa arr[i], actualiza min/max/acumulador
146     ; comensar i con +
147     jmp     .compute_stats_loop_final ; si i >= n, termino el recorrido -> salir del ciclo
148     movss   [rdi+8], [rdi+4] ; i = arr[i]
149     minss   xmm0, xmm0        ; min = minimo(min, i)
150     movss   [rdi+12], xmm0     ; max = max(max, i)
151
152     ; ----- calculo de (x - mean)^2 -----
153     ; ----- inicio de la sustracción para la variancia -----
154     movss   [rdi+16], xmm0     ; x = x - mean
155     movss   [rdi+20], xmm0     ; x = x - mean^2
156     addss   xmm0, xmm0        ; acumulador_var += (x - mean)^2
157
158     inc     rax               ; i = i + 1
159     jmp     .compute_stats_loop_principal ; volver a evaluar la condición del ciclo
160
161     .compute_stats_loop_final:
162     ; fin del recorrido: escribir resultados finales ----
163     movss   [rdi+24], xmm0     ; *mean_ptr = media encontrada
164     movss   [rdi+28], xmm0     ; *var_ptr = variancia encontrada
165     movss   [rdi+32], xmm0     ; *min_ptr = min encontrado
166     movss   [rdi+36], xmm0     ; *max_ptr = max encontrado
167     ; salir al epilog, saltándose el bloque de "arreglo vacío"
168

```

```

169     .compute_stats_arreglo_vacio:
170     ; ----- caso borde: n <= 0, todos los salidos se ponen en 0.0 -----
171     movss   [rdi], xmm0
172     movss   [rdi+4], xmm0
173     movss   [rdi+8], xmm0
174     movss   [rdi+12], xmm0
175     movss   [rdi+16], xmm0
176     movss   [rdi+20], xmm0
177
178     .compute_stats_retorno:
179     ; restaurar registros callee-saved y retornar ----
180     pop     r15
181     pop     r14
182     pop     r13
183     pop     r12
184     ret
185

```

(a) Prólogo y recepción de parámetros.

(b) Media y bucle principal (min, max, varian-
za).

(c) Caso borde de arreglo vacío y retorno.

Figura 3: Implementación de compute_stats escalar.

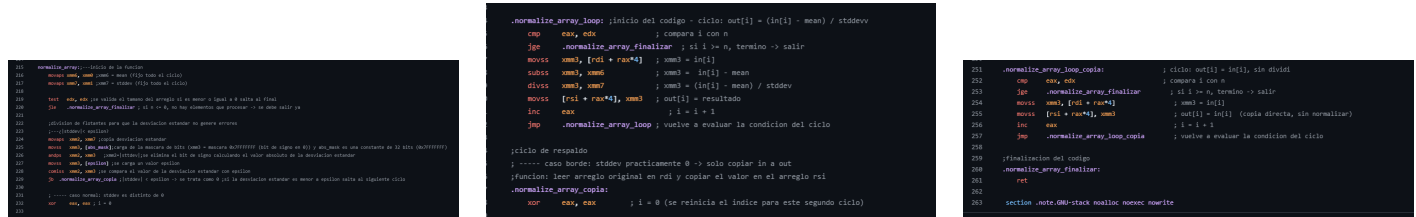
Al inicio de la función compute_stats se preservan los registros RBX, RBP, R12, R13, R14 y R15, ya que la función los conservar durante la llamada interna a sum_array. Los parámetros recibidos por la ABI se copian a estos registros: RBX guarda el puntero al arreglo (arr), EBP almacena n , mientras que los registros R12, R13, R14 y R15 guardan las direcciones de salida mean_ptr, var_ptr, min_ptr y max_ptr. Inmediatamente se valida el caso borde mediante test ebp, ebp y un salto condicional jle hacia .compute_stats_arreglo_vacio si $n \leq 0$.

Para calcular la media, se reutiliza sum_array pasándole nuevamente arr y n , se convierte n a punto flotante con cvtsi2ss y se divide la suma obtenida entre n mediante divss, escribiendo el resultado en *mean_ptr y resguardando una copia de la media en el registro XMM6 para su uso posterior. A continuación, un único recorrido escalar (.compute_stats_loop_principal) procesa el arreglo elemento por elemento: en cada iteración se actualizan el mínimo y el máximo acumulados mediante minss y maxss, y simultáneamente se calcula la desviación cuadrática $(x_i - \mu)^2$ con subss y mulss, acumulándola en el registro XMM5 para la

varianza. Al finalizar el recorrido, se escriben min y max en sus respectivas direcciones, se convierte n a flotante y se divide el acumulado entre n para obtener la varianza, que se almacena en `*var_ptr`.

Ahora bien, en el caso borde ($n \leq 0$), la etiqueta `.compute_stats_arreglo_vacio` limpia XMM0 con `xorps` y escribe 0.0 en las cuatro direcciones de salida (mean, var, min y max). Finalmente, `.compute_stats_retornar` restaura en orden inverso los registros callee-saved (R15, R14, R13, R12, RBP, RBX) antes de retornar el control al llamador, garantizando así el cumplimiento estricto de la ABI System V.

Finalmente, se muestran las imágenes para la función `normalize_array` versión escalar:



(a) Setup y validación de $\sigma \approx 0$.

(b) Ciclo principal de normalización.

(c) Caso borde: copia directa y finalización.

Figura 4: Código para la implementación de `normalize_array` escalar.

Al entrar a `normalize_array`, la media y la desviación estándar recibidas en XMM0 y XMM1 se guardan en XMM6 y XMM7, para mantenerlas fijas durante todo el ciclo. Tras validar que $n > 0$ mediante `test edx, edx` (saltando a `.normalize_array_finalizar` en caso contrario), se calcula el valor absoluto de la desviación estándar aplicando una máscara de bits (`abs_mask`) sobre una copia de XMM7, y se compara dicho valor absoluto contra una constante epsilon mediante `comiss`. Si $|\sigma| < \epsilon$, el flujo se desvía mediante `jb` hacia `.normalize_array_copia` para evitar una división por cero.

En el caso normal, `.normalize_array_loop` procesa el arreglo de uno en uno: para cada elemento se carga `in[i]` con `movss`, se le resta la media con `subss`, se divide entre la desviación estándar con `divss` y el resultado se escribe en `out[i]`, incrementando el índice hasta cubrir los n elementos. Cuando $\sigma \approx 0$, `.normalize_array_loop_copia` realiza en su lugar una copia directa elemento por elemento de `in` a `out` sin aplicar ninguna operación aritmética, evitando así indeterminaciones numéricas. Ambos ciclos convergen en la etiqueta `.normalize_array_finalizar`, donde la función retorna sin necesidad de limpiar registros vectoriales.

IV-B. Kernel vectorial

A continuación se muestran las imágenes del algoritmo relacionado a la función `sum_array` con su debida explicación:



(a) Inicialización y bucle vectorial.

(b) Reducción horizontal y residuo.

Figura 5: Código para la implementación de `sum_array` vectorial.

Primero se calcula la cantidad de bloques de 256 bits y el residuo mediante `div`. Si $N > 8$, el bucle `.vector_sum` acumula bloques de 8 flotantes en `ymm0` usando `vmovaps` (aprovechando la alineación provista por el driver) y `vaddps`. Posteriormente, `.horizontal_sum` reduce los acumuladores extrayendo las mitades de 128 bits con `vextractf128` para sumar vertical y horizontalmente (`vhaddps`) en `xmm1`. Finalmente, `.remainder_sum` procesa individualmente los elementos restantes, se transfiere el resultado a `xmm0` respetando la ABI System V y se limpia la parte alta de los registros YMM antes de retornar.

Ahora, se muestran las imágenes del algoritmo relacionado a la función `compute_stats`:

```

compute_stats:
; Teniendo en cuenta el orden de los argumentos de la funcion y el System V AMD64 ABI. Los datos se guardan en los registros
;RDI = arr
;RSI = n
;XMM0 = sum
;RDX = &mean
;RCX = &var
;R8 = &min
;R9 = &max

.stack_pointer_save: ;se agrega para evitar el segmentation fault existente al modificar los registros de los punteros
sub rsp, 40 ; se reservan 40 Bytes del stack
mov [rsp+0], rdx ; [rsp+0] = &mean
mov [rsp+8], rcx ; [rsp+8] = &var
mov [rsp+16], r8 ; [rsp+16] = &min
mov [rsp+24], r9 ; [rsp+24] = &max
mov [rsp+32], rdi ; arr

cmp esi, 0 ;se revisa si n=0
je .neqz

.mean:
cvtsi2ss xmm1, esi ;convierto n a un float para poder operar
vdivss xmm0, xmm0, xmm1 ;divido la suma total entre n

mov rax, [rsp+0] ; Carga en RAX la dirección donde está almacenada la variable mean.
vmovss [rax], xmm0 ; Pone xmm0 en el contenido de la dirección a la que apunta rax

```

(a) Inicio y media.

```

.variance:
.set_up:
vxorps ymm1, ymm1, ymm1 ;Acumulador para el calculo del termino cuadratico vectorialmente, tambien limpia xmm1

mov eax, esi
xor edx, edx
mov ecx, 8
div ecx ; calculamos catidad de bloques (eax) y sobrantes (edx)

mov r10d, 0
mov r11d, 0 ;contadores en cero
cmp eax, 0
je .quadratic_term_remainder ;si no se deben ejecutar bloques n<8 de una vez se procesa solo el sobrante

vbroadcastss ymm4, xmm0 ;llenamos todo el vector con el promedio para el calculo
.quadratic_term_vectorial:
vmovaps ymm2, [rdi] ;cargamos 8 numeros
vsubps ymm3, ymm2, ymm4 ;restamos xi-u
vmulps ymm2, ymm3, ymm3 ;elevamos al cuadrado
vaddps ymm1, ymm1, ymm2

add r10d, 1 ;sumo 1 al contador de bloques
add rdi, 32 ;voy al siguiente bloque del array

cmp r10d, eax ; si no igualo la cantidad de bloques totales sigo reduciendo
jne .quadratic_term_vectorial

.horizontal_sum:
vextractf128 xmm2, ymm1, 1 ;extraigo la parte alta de ymm1
vextractf128 xmm3, ymm1, 0 ;extraigo la parte baja de ymm1
vaddps xmm1, xmm3, xmm2
vhaddps xmm1, xmm1, xmm1
vhaddps xmm1, xmm1, xmm1 ; xmm1 = [(A-u)^2 + (B-u)^2 + ..., 0, 0, 0]
vxorps xmm2, xmm2, xmm2 ;se limpia el registro xmm2 para las operaciones con el residuo

```

(b) Varianza vectorial.

```

cmp edx, 0
je .variance_calculation ;si no hay residuo termine el calculo solo con la reduccion

.quadratic_term_remainder:
vmovss xmm3, [rdi] ; XMM3 = xi
vsubss xmm2, xmm3, xmm0 ;dato individual - media
vmulss xmm2, xmm2, xmm2 ;elevo al cuadrado
vaddss xmm1, xmm1, xmm2

add r11d, 1 ;sumo 1 al contador de residuo
add rdi, 4 ;voy al siguiente numero del array

cmp r11d, edx
jne .quadratic_term_remainder

.variance_calculation:
cvtsi2ss xmm4, esi ;convierto n a un float para poder operar en xmm4
vdivss xmm1, xmm1, xmm4 ;divido la suma total entre n

mov rax, [rsp+8] ; Carga en RAX la dirección donde está almacenada la variable var.
vmovss [rax], xmm1 ; Guarda el valor float de xmm1 en la dirección apuntada por RAX.

```

(c) Residuo de varianza.

Figura 6: Implementación de la media y la varianza en compute_stats.

Para preservar las direcciones recibidas por referencia (&mean, &var, &min, &max), se reservan 40 bytes en la pila y se verifica si el tamaño n es cero, desviando a .neqz si el arreglo está vacío. En .mean se calcula y almacena la media dividiendo la suma entre n mediante vdivss. Para la varianza, se dividen los bloques en .set_up y se duplica la media en ymm4 con vbroadcastss. El bucle .quadratic_term_vectorial resta la media, eleva al cuadrado (vmulps) y acumula las desviaciones por bloque. Tras reducir vectorialmente el resultado con vextractf128 y vhaddps, .quadratic_term_remainder acumula escalarmente los elementos del residuo antes de calcular la varianza final (vdivss) y escribirla en memoria.

```

.min:
.min_setup:
    mov rdi, [rsp+32] ;restauro dir de memoria
    mov eax, esi
    xor edx, edx
    mov ecx, 8
    div ecx

    cmp eax,0
    je .min_remainder_no_blocks ;si no hay bloques solo proceso residuo sin bloques
    vmovaps ymm1, [rdi] ;Cargo los primeros 8 nums en el registro base donde almaceno comparaciones
    mov r10d, 1 ;Ya se encuentra cargado el primer bloque (contador empieza en 1)
    xor r11d, r11d ;contador de sobrante en cero
    add rdi, 32 ;paso al siguiente bloque
    cmp r10d, eax
    je .horizontal_min ;si ya se proceso el unico bloque salto a reduccion horizontal

.min_vectorial:
    vmovaps ymm2, [rdi] ;cargamos 8 numeros a otro reg
    vminps ymm1, ymm1, ymm2 ;comparamos los 8 numeros del acumulador con los 8 nuevos
    add r10d, 1 ;sumo 1 al contador de bloques
    add rdi, 32 ;voy al siguiente bloque del array
    cmp r10d, eax ;si no igualo la cantidad de bloques totales sigo comparando vectorialmente
    jne .min_vectorial

.horizontal_min:
    vextractf128 xmm2, ymm1, 1 ;extraigo la parte alta
    vextractf128 xmm3, ymm1, 0 ;extraigo la parte baja
    vminps xmm1, xmm2, xmm3 ;comparo el min de parte alta y baja
    vshufps xmm2, xmm1, xmm1, 0x4E ;Hago un shuffle para comparar los numeros con otros
    vminps xmm1, xmm1, xmm2 ;comparo los numeros
    vshufps xmm2, xmm1, xmm1, 0xB1 ;repito
    vminps xmm1, xmm1, xmm2
    ;En este punto xmm1[0] contiene el minimo de los bloques

    cmp edx,0
    je .min_return ;si no hay residuo retorno

```

(a) Mínimo vectorial.

```

.min_remainder_blocks:
.block_loop_min:
    vmovss xmm2, [rdi] ;cargo un numero individual
    vminss xmm1, xmm1, xmm2 ;comparo el numero con el minimo acumulado
    add r11d, 1 ;sumo 1 al contador de residuo
    add rdi, 4 ;me muevo a la posicion del siguiente residuo
    cmp r11d, edx ;si igualo la cantidad de sobrantes termino
    jne .block_loop_min

    jmp .min_return

.min_remainder_no_blocks:
    xor r11d, r11d ;contador de sobrante en cero
    vmovss xmm1, [rdi] ;cargo el primer numero
    add rdi, 4 ;me muevo al siguiente numero
    add r11d, 1 ;ya se proceso el primer numero
.no_block_loop_min:
    cmp r11d, edx
    je .min_return
    vmovss xmm2, [rdi] ;cargo el siguiente numero
    vminss xmm1, xmm1, xmm2 ;comparo los numeros
    add r11d, 1
    add rdi, 4 ;me muevo a la posicion del siguiente numero
    jmp .no_block_loop_min

.min_return:
    mov rax, [rsp+16] ;cargo la direccion donde se almacenara el minimo
    vmovss [rax], xmm1 ;guardo el minimo en la direccion correspondiente

```

(b) Residuo de mínimo.

Figura 7: Implementación del cálculo del valor mínimo en compute_stats.

El cálculo del mínimo toma los primeros 8 elementos como semilla en ymm1 y compara vectorialmente los bloques restantes con vminps. En .horizontal_min se descompone el registro a 128 bits y se aplican permutaciones (vshufps) con vminps hasta aislar el mínimo escalar. El residuo se compara mediante vminss, usando .block_loop_min para elementos restantes o .min_remainder_no_blocks si $n < 8$, escribiendo el resultado final en &min.

```

.max: ;exactamente la misma logica que el minimo pero con las funciones de maximo y la direccion final correcta
.max_setup:
    mov rdi, [rsp+32]
    mov eax, esi
    xor edx, edx
    mov ecx, 8
    div ecx

    cmp eax, 0
    je .max_remainder_no_blocks
    vmovaps ymm1, [rdi]
    mov r10d, 1
    xor r11d, r11d
    add rdi, 32
    cmp r10d, eax
    je .horizontal_max

.max_vectorial:
    vmovaps ymm2, [rdi]
    vmaxps ymm1, ymm1, ymm2
    add r10d, 1
    add rdi, 32
    cmp r10d, eax
    jne .max_vectorial

.horizontal_max:
    vextractf128 xmm2, ymm1, 1
    vextractf128 xmm3, ymm1, 0
    vmaxps xmm1, xmm2, xmm3
    vshufps xmm2, xmm1, xmm1, 0x4E
    vmaxps xmm1, xmm1, xmm2
    vshufps xmm2, xmm1, xmm1, 0xB1
    vmaxps xmm1, xmm1, xmm2

    cmp edx, 0
    je .max_return

```

(a) Máximo vectorial.

```

.max_remainder_blocks:
    .block_loop_max:
        vmovss xmm2, [rdi]
        vmaxss xmm1, xmm1, xmm2
        add r11d, 1
        add rdi, 4
        cmp r11d, edx
        jne .block_loop_max

    jmp .max_return

.max_remainder_no_blocks:
    xor r11d, r11d
    vmovss xmm1, [rdi]
    add rdi, 4
    add r11d, 1
    .no_block_loop_max:
        cmp r11d, edx
        je .max_return
        vmovss xmm2, [rdi]
        vmaxss xmm1, xmm1, xmm2
        add r11d, 1
        add rdi, 4
        jmp .no_block_loop_max

.max_return:
    mov rax, [rsp+24]
    vmovss [rax], xmm1

```

(b) Residuo de máximo.

Figura 8: Implementación del cálculo del valor máximo en compute_stats.

El cálculo del máximo sigue exactamente la misma estructura que el mínimo, sustituyendo las instrucciones de comparación por sus equivalentes de valor máximo (vmaxps para bloques vectoriales, vmaxss para residuos y permutaciones adaptadas). El valor resultante se escribe en la dirección &max previamente guardada en la pila.

```

.rsp_original_position_return:
    add rsp, 40 ;Se devuelve rsp a la posicon original para evitar el segmentation fault
    vzeroupper
    ret

.neqz: ;Caso borde n=0. Simplemente no hay datos se pone todo en cero
    vxorps xmm0, xmm0, xmm0
    mov rax, [rsp+0]
    vmovss [rax], xmm0
    mov rax, [rsp+8]
    vmovss [rax], xmm0
    mov rax, [rsp+16]
    vmovss [rax], xmm0
    mov rax, [rsp+24]
    vmovss [rax], xmm0
    jmp .rsp_original_position_return

```

Figura 9: Return y gestión de casos borde en compute_stats.

En la salida normal, se restaura la pila (add rsp, 40) y se ejecuta vzeroupper para evitar degradaciones de rendimiento SIMD. En el caso borde de arreglo vacío ($n = 0$), la etiqueta .neqz escribe valores nulos en todas las referencias enviadas y finaliza la ejecución.

Finalmente, se muestran las imágenes para la función normalize_array:

```

normalize_array:
rdi = in, rsi = out, edx = n, xmm0 = mean, xmm1 = stddev
; caso borde: si stddev == 0.0, copie in[i] en out[i] tal cual.
; out[i] = (in[i] - mean) / stddev

; caso borde, no hay elementos n=0
cmp edx, 0
je .end

; set_up: se cargan en registros la media, stddev (precarga de valores) y se calcula la cantidad de bloques y residuo, se reinician contadores
mov eax, edx
xor edx, edx
mov ecx, 0
div ecx ; ecx = [edx:eax] // ecx y edx = [edx:eax] mod ecx
mov rdi, 0 ; contador bloque
mov rsi, 0 ; contador residuo

; stddev_eqz:
vzeroq xmm0, xmm0
vucomiss xmm1, xmm0 ; stddev=0?
je .write_values_border_case

; block_eqz:
cmp eax, 0
je .normalize_remainder

```

(a) Setup y validación.

```

; normalize_blocks:
vbroadcastss ymm2, xmm0 ; ymm2 buffer con medias
vbroadcastss ymm3, xmm1 ; ymm3 buffer con stdevs
vmovaps ymm5, [rdi] ; ymm5 acumulador resultados
vsubps ymm5, ymm5, ymm2
vdivps ymm5, ymm5, ymm3 ; operacion
vmovaps [rsi], ymm5

add rdi, 32
add rsi, 32
add rdi, 1
cmp rdi, eax
jne .normalize_blocks

; remainder_eqz:
cmp edx, 0
je .end

; normalize_remainder:
vmovss xmm5, [rdi] ; cargo in[i]
vsubss xmm5, xmm5, xmm0
vdivss xmm5, xmm5, xmm1
vmovss [rsi], xmm5 ; guardo in[i] en out[i]
add rdi, 4
add rsi, 4
add rdi, 1
cmp rdi, edx
jne .normalize_remainder
jmp .end

```

(b) Bloques y residuo.

```

; write_values_border_case:
cmp eax, 0
je .remainder_loop_bc

; block_loop_bc:
vmovaps ymm5, [rdi]
vmovaps [rsi], ymm5
add rdi, 32
add rsi, 32
add rdi, 1
cmp rdi, eax
jne .block_loop_bc

; remainder_loop_bc:
cmp edx, 0
je .end

vmovss xmm5, [rdi] ; cargo in[i]
vmovss [rsi], xmm5 ; guardo in[i] en out[i]
add rdi, 4
add rsi, 4
add rdi, 1
cmp rdi, edx
jne .remainder_loop_bc

; end:
vzeroupper
ret

```

(c) Caso $\sigma = 0$ y salida.

Figura 10: Código para la implementación de normalize_array vectorial.

Tras validar si el arreglo está vacío, .set_up calcula los bloques de 256 bits y valida mediante vucomiss si la desviación estándar es cero, desviando el flujo a .write_values_border_case para prevenir divisiones por cero.

En .normalize_blocks se duplican la media y la desviación en registros YMM (vbroadcastss) para restar (vsubps) y dividir (vdivps) en bloques de 8 elementos, mientras que .normalize_remainder atiende los elementos sobrantes mediante instrucciones escalares. Si $\sigma = 0$, .write_values_border_case realiza una copia directa del arreglo de entrada al de salida, concluyendo en .end con la limpieza de registros via vzeroupper.

El uso de AVX2 procesa 8 flotantes por ciclo, duplicando la capacidad respecto a SSE. Debido a la ausencia de reducciones horizontales directas en 256 bits, la estrategia descompone el registro en mitades de 128 bits (vextractf128) combinando sumas cruzadas (vhaddps) o permutaciones (vshufps) con comparaciones paralelas. Finalmente, el desacoplamiento mediante $n/8$ delega los remanentes ($n \bmod 8 \neq 0$) y arreglos pequeños a un procesamiento escalar de precisión individual, evitando padding, garantizando seguridad ante desviaciones nulas y limpiando el estado YMM antes del retorno.

IV-C. Implementación

Para implementar el código tanto escalar como vectorial, se deben correr las siguientes instrucciones solamente al inicio:

1. sudo apt update
2. sudo apt install -y build-essential nasm python3
3. git clone https://github.com/Ahenao16/Proyecto_1_Arqui.git

Ahora bien, se deben ejecutar las siguientes instrucciones, para correr el proyecto con diferentes casos de N (tamaño del arreglo de datos):

1. cd ruta/a/Proyecto_1_Arqui/proyecto_vectorial/proyecto_vectorial (ingresar a la carpeta del proyecto)
2. make clean
3. python3 tools/gen_input.py 1000003 data/input.dat random 42 (El numero 1000003 se debe variar para ir cambiando el N que se quiere analizar)
4. make run-scalar
5. make run-vector
6. python3 tools/verify_reference.py data/input.dat data/output_scalar.dat.stats.txt
7. python3 tools/verify_reference.py data/input.dat data/output_vector.dat.stats.txt

V. CASOS DE PRUEBA

A continuación se presenta la tabla comparativa con la cantidad de datos (N), el modo de generación de entrada, la salida esperada de referencia, la salida obtenida por cada versión y el resultado final de la verificación bajo el criterio de una tolerancia numérica con un valor de $1e-4$. Estas pruebas evaluaron el comportamiento con diferentes archivos de entrada para comprobar aspectos como el manejo de bloques de 8 números y el remanente de la versión vectorial, así como el correcto manejo de datos uno a uno de la versión escalar. Los casos $N = 0$ y $N = 1$ forzaron la ejecución exclusiva de la lógica de remanente o vacía, mientras que las pruebas con $N = 6$ y $N = 9$ validaron el procesamiento combinado de iteraciones vectoriales principales junto a su respectivo residuo. Adicionalmente, la prueba con $N = 100000$ evaluó la acumulación de error flotante y el correcto manejo numérico de ambos kernel con una cantidad de datos más significativa en comparación al resto de pruebas, así mismo se comprueban todo tipo de valores tanto positivos como negativos. Respecto a los modos de entrada, el modo random generó valores aleatorios en el rango $[-100, 100]$. El modo constant fijó todos los elementos en 5,0 ($N = 10$) para verificar que el sistema

procese adecuadamente casos donde la varianza y desviación estándar deben dar exactamente cero sin causar indeterminaciones matemáticas. El modo edge ($N = 9$) intercaló valores extremos ($\pm 10^6$), ceros y números muy cercanos a cero ($\pm 0,0001$) para comprobar la resistencia del algoritmo frente a underflow o overflow.

N	Modo	Versión	Salida Esperada (Referencia)	Salida Obtenida	Máximo error relativo reportado	Resultado
0	random	Escalar	Suma: 0.0, Media: 0.0, Var: 0.0, StdDev: 0.0, Min: 0.0, Max: 0.0	Suma: 0.0, Media: 0.0, Var: 0.0, StdDev: 0.0, Min: 0.0, Max: 0.0	0 (Todos)	PASA
		Vectorial	Suma: 0.0, Media: 0.0, Var: 0.0, StdDev: 0.0, Min: 0.0, Max: 0.0	Suma: 0.0, Media: 0.0, Var: 0.0, StdDev: 0.0, Min: 0.0, Max: 0.0	0 (Todos)	PASA
1	random	Escalar	Suma: -16.596714, Media: -16.596714, Var: 0.0, StdDev: 0.0, Min: -16.596714, Max: -16.596714	Suma: -16.596714, Media: -16.596714, Var: 0.0, StdDev: 0.0, Min: -16.596714, Max: -16.596714	$1,19152 \times 10^{-9}$ (sum, mean, min, max)	PASA
		Vectorial	Suma: -16.596714, Media: -16.596714, Var: 0.0, StdDev: 0.0, Min: -16.596714, Max: -16.596714	Suma: -16.596714, Media: -16.596714, Var: 0.0, StdDev: 0.0, Min: -16.596714, Max: -16.596714	$1,19152 \times 10^{-9}$ (sum, mean, min, max)	PASA
6	random	Escalar	Suma: -194.008766, Media: -32.334794, Var: 3106.906631, StdDev: 55.739632, Min: -94.702629, Max: 50.318546	Suma: -194.008759, Media: -32.334793, Var: 3106.906490, StdDev: 55.739632, Min: -94.702629, Max: 50.318546	$4,54432 \times 10^{-8}$ (var)	PASA
		Vectorial	Suma: -194.008766, Media: -32.334794, Var: 3106.906631, StdDev: 55.739632, Min: -94.702629, Max: 50.318546	Suma: -194.008759, Media: -32.334793, Var: 3106.906490, StdDev: 55.739632, Min: -94.702629, Max: 50.318546	$4,54432 \times 10^{-8}$ (var)	PASA
9	edge	Escalar	Suma: 0.0, Media: 0.0, Var: $4,444444 \times 10^{11}$, StdDev: 666666.666667, Min: -1000000.0, Max: 1000000.0	Suma: 0.0, Media: 0.0, Var: $4,444444 \times 10^{11}$, StdDev: 666666.688000, Min: -1000000.0, Max: 1000000.0	$3,19997 \times 10^{-8}$ (stddev)	PASA
		Vectorial	Suma: 0.0, Media: 0.0, Var: $4,444444 \times 10^{11}$, StdDev: 666666.666667, Min: -1000000.0, Max: 1000000.0	Suma: 0.0, Media: 0.0, Var: $4,444444 \times 10^{11}$, StdDev: 666666.688000, Min: -1000000.0, Max: 1000000.0	$3,19997 \times 10^{-8}$ (stddev)	PASA
10	constant	Escalar	Suma: 50.0, Media: 5.0, Var: 0.0, StdDev: 0.0, Min: 5.0, Max: 5.0	Suma: 50.0, Media: 5.0, Var: 0.0, StdDev: 0.0, Min: 5.0, Max: 5.0	0 (Todos)	PASA
		Vectorial	Suma: 50.0, Media: 5.0, Var: 0.0, StdDev: 0.0, Min: 5.0, Max: 5.0	Suma: 50.0, Media: 5.0, Var: 0.0, StdDev: 0.0, Min: 5.0, Max: 5.0	0 (Todos)	PASA
100 000	random	Escalar	Suma: -1335.995297, Media: -0.013360, Var: 3317.491485, StdDev: 57.597669, Min: -99.991783, Max: 99.998207	Suma: -1335.950930, Media: -0.013360, Var: 3317.444090, StdDev: 57.597259, Min: -99.991783, Max: 99.998207	$3,32087 \times 10^{-5}$ (sum)	PASA
		Vectorial	Suma: -1335.995297, Media: -0.013360, Var: 3317.491485, StdDev: 57.597669, Min: -99.991783, Max: 99.998207	Suma: -1335.944348, Media: -0.013359, Var: 3317.488048, StdDev: 57.597641, Min: -99.991783, Max: 99.998207	$3,81414 \times 10^{-5}$ (sum)	PASA

Tabla I: Resumen de pruebas de verificación.

VI. RESULTADOS DE RENDIMIENTO

Para medir la ganancia obtenida por la parte vectorial, se ejecutó un script de benchmark que, para cada tamaño N (desde 1 hasta 10 000 000 elementos, en potencias de 2 y algunos valores intermedios), corre ambos kernels sobre un número de repeticiones reps tal que $\text{reps} \times N \approx 50\,000\,000$ elementos procesados, garantizando de esta manera, una carga de trabajo comparable entre los tamaños de N pequeños y grandes. Ahora bien, de cada prueba se registra la media, la mediana, el mínimo y la desviación estándar del tiempo de ejecución de la versión escalar y la versión vectorial, y a partir de ellas se obtienen dos parámetros: el speedup mediano (tiempo escalar entre tiempo vectorial) y el costo normalizado en nanosegundos por elemento, que permite separar el costo fijo de invocación del costo de procesar cada dato. Las pruebas se ejecutaron sobre un procesador Intel Core i7-7700HQ (de 4 núcleos físicos, 8 hilos, y una frecuencia de hasta 3.8 GHz), con una jerarquía de caché de 32 KiB de L1d por núcleo, 256 KiB de L2 por núcleo y 6 MiB de L3 compartida, bajo el gobernador de frecuencia powersave. Se realizaron tres pruebas: una sin fijar el proceso a un núcleo específico (cpu_pin=ninguno), una fijándolo al núcleo 2 (cpu_pin=2) para reducir el ruido por entre núcleos, y una corrida de validación con solo tres valores de N (8, 1 000 y 1 000 000) y 20 repeticiones por punto, sin fijar núcleo.

VI-A. Tiempo de ejecución en función de N

La Figura 11 muestra el tiempo mediano de ejecución de sum_array para ambas versiones, con ambos ejes en escala logarítmica, correspondiente a la corrida sin fijar núcleo.

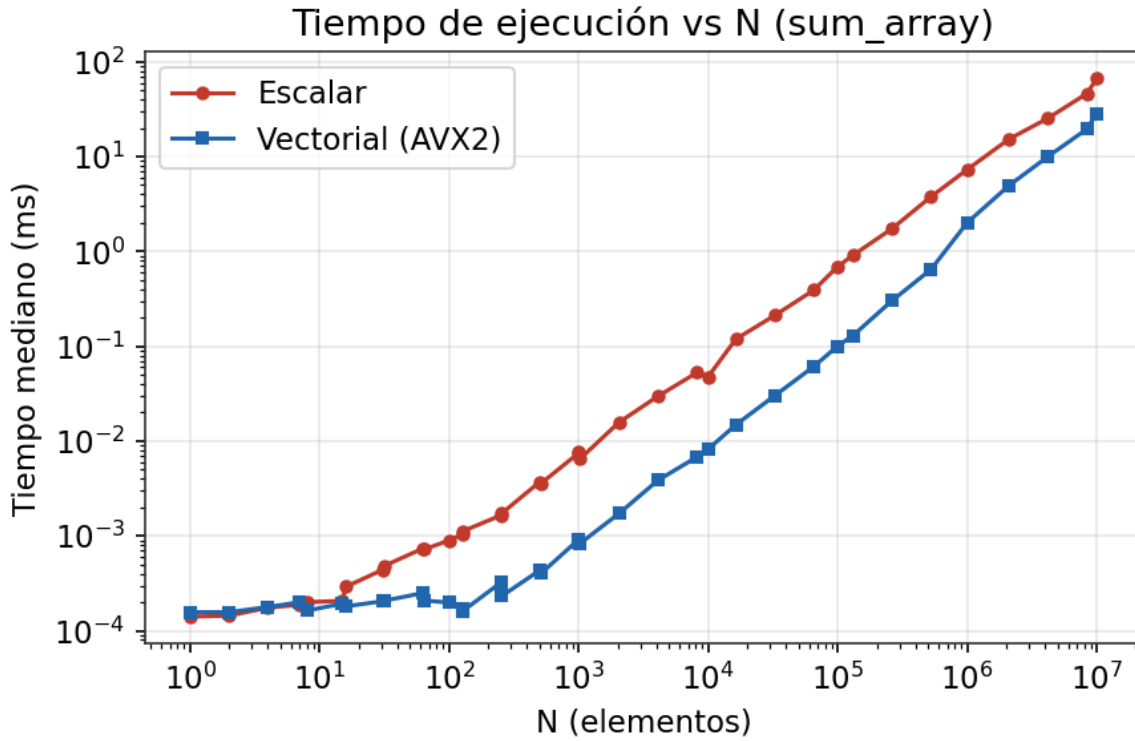


Figura 11: Tiempo mediano de ejecución vs N (escala log-log), corrida sin fijar núcleo.

Se observan dos regímenes claramente diferenciados. Para N pequeño (hasta unas pocas decenas de elementos) el tiempo de ejecución prácticamente no depende de N : domina el costo fijo de invocar la rutina, que en la versión vectorial incluye además la inicialización de los registros YMM y la lógica de división para calcular bloques y remanente. Esto explica que en $N = 1$ la versión vectorial resulte más lenta por elemento (157.5 ns/elemento) que la escalar (141.0 ns/elemento): el trabajo real es mínimo y solo se paga el sobrecosto de arranque. A partir de decenas o cientos de elementos, el trabajo por hacer empieza a dominar sobre el costo fijo y ambas curvas adoptan una pendiente constante en el log-log, consistente con un algoritmo $O(N)$. En ese tramo la curva vectorial queda claramente por debajo de la escalar, evidenciando la ganancia de procesar 8 elementos por instrucción con AVX2 frente a uno solo a la vez.

VI-B. Speedup y efecto de la jerarquía de caché

La Figura 12 presenta el speedup mediano (tiempo escalar entre tiempo vectorial) en función de N , con líneas verticales punteadas en los valores de N a partir de los cuales el conjunto de datos ($N \times 8$ bytes, un float de entrada y uno de salida por elemento) deja de caber en cada nivel de caché: $N = 8\,192$ para L1 (32 KiB), $N = 65\,536$ para L2 (256 KiB) y $N = 1\,000\,000$ para L3 (6 MiB).

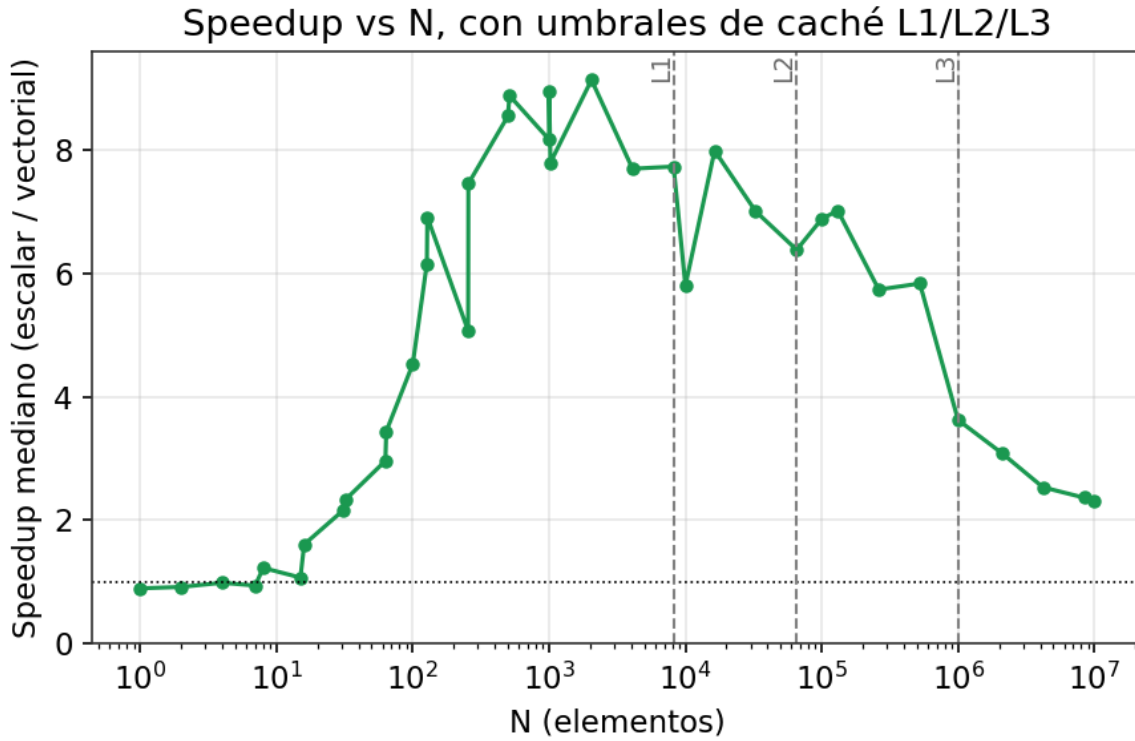


Figura 12: Speedup mediano vs N , con umbrales de caché L1/L2/L3, corrida sin fijar núcleo.

Para N muy pequeño el speedup es menor a $1\times$ ($0,895\times$ en $N = 1$), ya que el sobrecosto de la versión vectorial no alcanza a compensarse con pocos elementos. El speedup crece rápidamente y alcanza su máximo, $9,14\times$, en $N = 2048$, mientras que los datos todavía caben holgadamente en L1/L2, donde el acceso a memoria es rápido y el cuello de botella real es el cómputo, terreno favorable para SIMD. Al cruzar el umbral de L1 ($N = 8192$) el speedup cae a $7,74\times$, y al cruzar el de L2 ($N = 65536$) baja a $6,39\times$; la caída se acentúa después de superar L3 ($N = 1000000$), donde el speedup es de $3,63\times$, hasta llegar a $2,32\times$ en $N = 10000000$. La Tabla II resume estos puntos de referencia.

Tabla II: Speedup mediano en los umbrales de caché (prueba sin fijar núcleo).

Punto de referencia	N	Speedup
Overhead dominante	1	$0,90\times$
Máximo speedup (dentro de L1/L2)	2048	$9,14\times$
Umbral L1 (32 KiB)	8192	$7,74\times$
Umbral L2 (256 KiB)	65536	$6,39\times$
Umbral L3 (6 MiB)	1000000	$3,63\times$
N más grande evaluado	10000000	$2,32\times$

Este comportamiento confirma que, mientras los datos caben en caché, la vectorización aprovecha al máximo el paralelismo de datos y el tiempo está limitado por el cómputo. Una vez que el arreglo excede la capacidad de todos los niveles de caché, tanto la versión escalar como la vectorial quedan limitadas por el ancho de banda hacia la memoria RAM, de modo que la vectorización deja de procesar los datos más rápido de lo que la memoria logra suministrarlos, y el speedup se aleja del ideal teórico de $8\times$ tanto por este efecto, como por la ley de Amdahl.

VI-C. Costo normalizado por elemento

La Figura 13 normaliza el tiempo total entre la cantidad de elementos procesados (ns/elemento), lo que separa el costo fijo de invocación del costo real de procesamiento por dato.

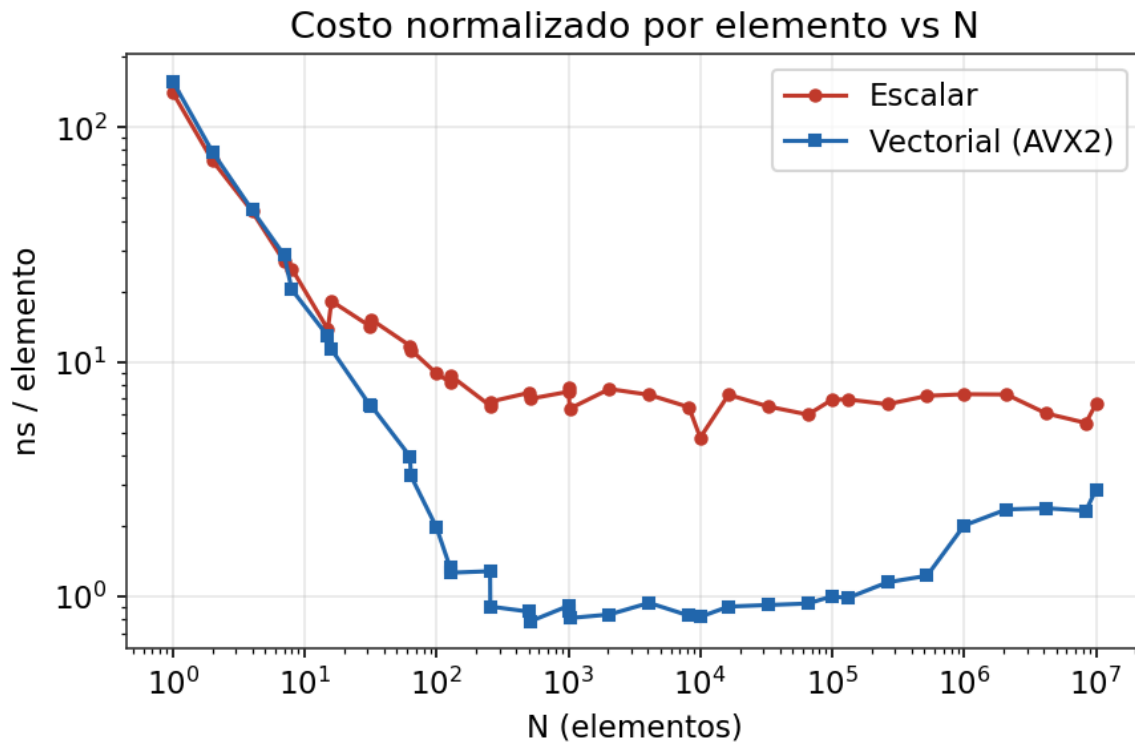


Figura 13: Costo normalizado (ns/elemento) vs N , corrida sin fijar núcleo.

Para N pequeños el costo por elemento es alto en ambas versiones (141.0 ns/elemento escalar y 157.5 ns/elemento vectorial en $N = 1$), porque el costo fijo de arrancar la rutina se reparte entre muy pocos elementos. A medida que N crece, el costo fijo decae y el ns/elemento cae hasta estabilizarse en un mínimo que representa el costo real de procesar un elemento ya amortizado el overhead: aproximadamente 4,77 ns/elemento para la versión escalar (en $N = 10\,000$) y 0,78 ns/elemento para la vectorial (en $N = 512$), una razón cercana a $6\times$, coincidiendo con el speedup medido en esa región. Para N muy grandes el costo por elemento vuelve a subir (6.63 ns/elemento escalar y 2.85 ns/elemento vectorial en $N = 10\,000\,000$), confirmando desde otro ángulo lo observado en la Figura 12: al salir de todos los niveles de caché, cada elemento tarda más en procesarse porque debe esperarse a que llegue desde la memoria principal, y ese efecto perjudica a la versión vectorial. Que sin el cuello de botella de memoria es capaz de procesar 8 elementos por ciclo en comparación con la escalar.

VI-D. Efecto de fijar el proceso a un núcleo

Para evaluar el impacto del ruido provocado por la migración del proceso entre los núcleos del sistema operativo, se repitió el barrido completo de N fijando el proceso al núcleo 2 mediante `taskset (cpu_pin=2)`. La Figura 14 superpone el speedup mediano de ambas pruebas.

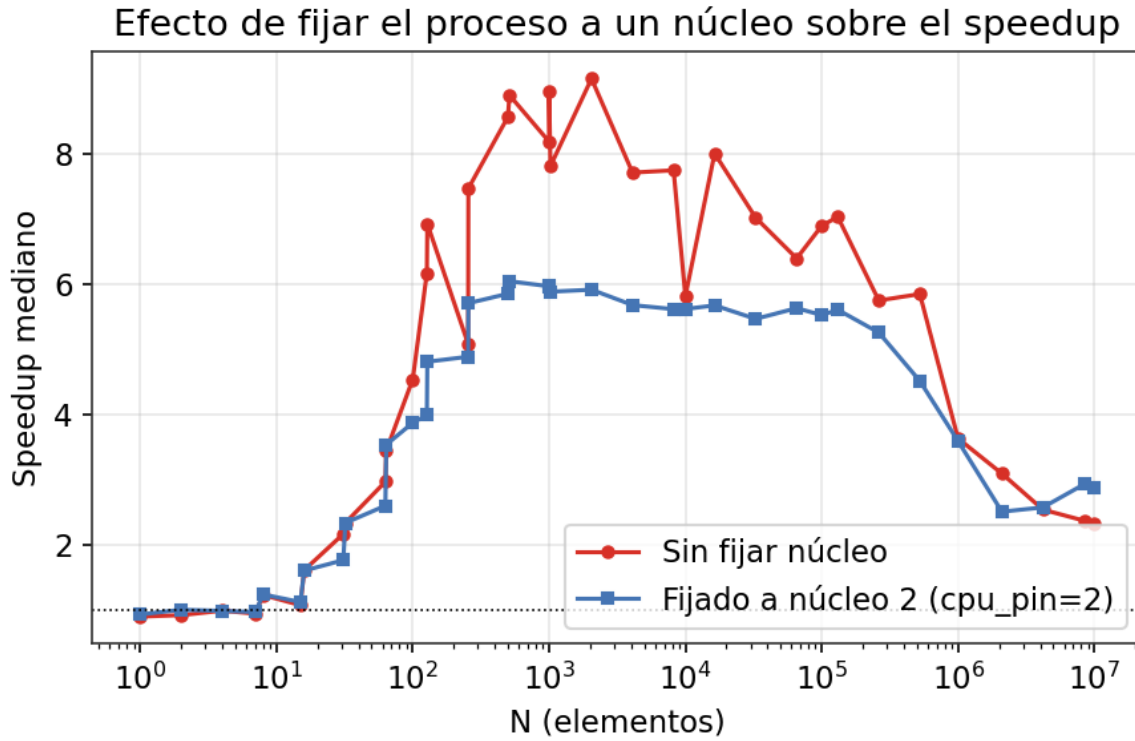


Figura 14: Comparación del speedup mediano con y sin fijación de núcleo.

La prueba sin fijar núcleo presenta oscilaciones marcadas (por ejemplo, caídas puntuales en $N = 127$ y $N = 10\,000$) atribuibles a la migración del proceso entre los cuatro núcleos físicos y ocho hilos lógicos del i7-7700HQ, así como a variaciones en el escalado de frecuencia bajo el gobernador powersave (26 % de uso de escalado reportado en esa corrida, frente a 58 % en la corrida fijada). La prueba con `cpu_pin=2` produce una curva notablemente más suave, con un máximo de speedup de $6,04\times$ en $N = 512$ y valores en los umbrales de caché de $5,61\times$ (L1, $N = 8\,192$), $5,63\times$ (L2, $N = 65\,536$) y $3,59\times$ (L3, $N = 1\,000\,000$), cayendo a $2,87\times$ en $N = 10\,000\,000$. Una tendencia cualitativa idéntica a la de la corrida sin fijar el núcleo, pero con menor varianza punto a punto. Los tiempos absolutos de la prueba fijada son menores (p. ej., 45.0 ns/elemento escalar en $N = 1$ frente a 141.0 ns/elemento sin fijar), lo cual indica que, evitar la migración entre núcleos no solamente reduce el ruido de la medición, sino que también permite un mejor aprovechamiento de la caché local del núcleo y menor penalización por cambios de contexto. En consecuencia, aunque el speedup máximo observado resultó más alto en la prueba sin pin ($9,14\times$ vs. $6,04\times$), este valor debe interpretarse con atención:, ya que refleja en parte picos favorables de calendarización más que una diferencia real de eficiencia entre ambos escenarios de ejecución.

VI-E. Validación con muestreo reducido de N

Como verificación adicional, se ejecutó una prueba independiente con solo tres valores de N (8, 1 000 y 1 000 000) y 20 repeticiones por punto, sin fijar núcleo. La Tabla III compara estos resultados contra los obtenidos en el barrido completo para los mismos valores de N .

Tabla III: Validación cruzada del speedup en tres valores de N .

N	Speedup (barrido completo)	Speedup (3 puntos)	Diferencia
8	1,23×	1,21×	1,6 %
1 000	8,18×	8,12×	0,8 %
1 000 000	3,63×	3,56×	1,8 %

La concordancia entre ambas pruebas independientes (diferencias inferiores al 2 %) da confianza en cuanto a la consistencia de las mediciones. No obstante, con solo tres puntos y sin fijar núcleo se observó una dispersión considerable entre repeticiones individuales: en $N = 1\,000$ la desviación estándar del tiempo vectorial alcanzó cerca del 42 % de su media, y en $N = 1\,000\,000$ la escalar varió aproximadamente 16 % y la vectorial 44 %, lo cual es consistente con la interferencia del planificador del sistema operativo al mover el proceso entre núcleos durante la ejecución, y refuerza la conveniencia de reportar la mediana (más robusta a valores atípicos) por encima de la media como métrica principal de tiempo.

VI-F. Discusión general

En conjunto, los resultados muestran que la vectorización con AVX2 logra ganancias sustanciales (hasta $\sim 9\times$) mientras el conjunto de datos permanece en la caché L1/L2, un régimen limitado por cómputo donde procesar 8 floats por instrucción se traduce directamente en menos ciclos de reloj. A medida que N crece más allá de la capacidad de L3, el sistema pasa a estar limitado por el ancho de banda de memoria principal, y el speedup converge hacia valores de entre $2\times$ y $3\times$, muy por debajo del factor teórico de $8\times$. Este comportamiento corresponde al que se menciona en el apartado de metodología, al mencionar la ley de Amdahl y el ancho de banda de memoria como los dos factores que impiden alcanzar la aceleración ideal a nivel práctico.

VII. SESIONES DE DEPURACIÓN (GDB)

A continuación se documentan dos sesiones de depuración en GDB realizadas sobre el ejecutable vectorial `norm_vector`, con el fin de inspeccionar el comportamiento de los registros YMM y verificar el correcto flujo de ejecución en ensamblador AVX2.

VII-A. Sesión 1: Inspección de `sum_array` y Acumulación en Registros YMM

```
alejandro@henao:~/Proyecto_1_Arquitectura_vectorial/proyecto_vectorial$ gdb ./bin/norm_vector
GNU gdb (Ubuntu 17.1-2ubuntu1) 17.1
Copyright (C) 2015 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./bin/norm_vector...
(gdb) set args data/input.dat data/output.dat 1
(gdb) b main
Breakpoint 1 at 0x1138: file src/driver.c, line 126.
(gdb) run
Starting program: /home/alejandro/henao/Proyecto_1_Arquitectura_vectorial/proyecto_vectorial/bin/norm_vector data/input.dat data/output.dat 1

This GDB supports auto-downloading debuginfo from the following URLs:
<http://sourceware.org/bugzilla/bugzilla.cgi?bugzilla=1>
Enable debuginfo for this session? (y or n) y
Debuginfo has been enabled.
To make this setting permanent, add 'set debuginfo enabled on' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
```

Figura 15: Inicialización de la sesión de depuración en GDB.

En la Figura 15 se inicia el depurador GDB cargando el binario `./bin/norm_vector`. Se definen los argumentos del programa mediante `set args data/input.dat data/output.dat 1`, se establece un punto de interrupción en la función principal (`b main`) y se inicia la ejecución con el comando `run`.

```
Breakpoint 1, main (argc=4, argv=0x7ffffffdcf8) at src/driver.c:126
126      main (argc, argv) {
(gdb) x/15i sum_array
0x55555555940: <sum_array>: mov     %esi,%eax
0x55555555942: <sum_array+2>: xor     %edx,%edx
0x55555555944: <sum_array+4>: mov     $0x0,%ecx
0x55555555946: <sum_array+6>: div     %ecx
0x55555555948: <sum_array+8>: mov     $0x0,%r8d
0x5555555594a: <sum_array+10>: mov     $0x0,%r9d
0x5555555594c: <sum_array+12>: vxorps  %ymm0,%ymm0,%ymm0
0x5555555594e: <sum_array+14>: vxorps  %xmm0,%xmm0,%xmm0
0x55555555950: <sum_array+16>: cmp     $0x0,%eax
0x55555555952: <sum_array+18>: je      0x55555555991 <sum_array.remainder_sum>
0x55555555954: <sum_array.vector_sum>: vmovaps (%rdi),%ymm1
0x55555555956: <sum_array.vector_sum+2>: vaddps %ymm1,%ymm0,%ymm0
0x55555555958: <sum_array.vector_sum+4>: add     $0x1,%r8d
0x5555555595a: <sum_array.vector_sum+6>: add     $0x2,%rdi
0x5555555595c: <sum_array.vector_sum+8>: cmp     %eax,%r8d
(gdb) b *0x55555555958
Breakpoint 2 at 0x55555555958: file asm/vector/stats_vector.asm, line 24.
(gdb) continue
Continuing.
Breakpoint 2, sum_array.vector_sum () at asm/vector/stats_vector.asm:24
24      vaddps %ymm1,%ymm0,%ymm0 ; suma el bloque en %ymm1 en el acumulador %ymm0
```

Figura 16: Desensamblado de `sum_array` y punto de parada en la suma vectorial.

En la Figura 16 se desensamblan las primeras instrucciones de la función `sum_array` con `x/15i sum_array`. Se coloca un breakpoint en la instrucción de acumulación vectorial `vaddps` (`b *0x55555555958`) en la línea 24 de `stats_vector.asm`.

```

(gdb) p/f $ymm1.v8_float
$1 = {57.6478844, 9.69173527, -57.384346, -75.4025345, -8.09004307, 21.5168056, 26.1001949, -11.5577393}
(gdb) p/f $ymm0.v8_float
$2 = {0, 0, 0, 0, 0, 0, 0, 0}
(gdb) stepi
26      add r1d, 1 ;sumo 1 al contador de bloques
(gdb) p/f $ymm1.v8_float
$3 = {57.6478844, 9.69173527, -57.384346, -75.4025345, -8.09004307, 21.5168056, 26.1001949, -11.5577393}
(gdb) p/f $ymm0.v8_float
$4 = {57.6478844, 9.69173527, -57.384346, -75.4025345, -8.09004307, 21.5168056, 26.1001949, -11.5577393}
(gdb) continue
Continuing.

Breakpoint 2, sum_array.vector_sum () at asm/vector/stats_vector.asm:24
24      vaddps ymm0, ymm0, ymm1 ;sumo el bloque en ymm0 en el acumulador ymm0
(gdb) p/f $ymm1.v8_float
$5 = {-39.5025101, -12.5138882, 97.5386276, 98.0559158, -19.9952335, 88.0046082, 38.1431694, 52.1846088}
(gdb) p/f $ymm0.v8_float
$6 = {57.6478844, 9.69173527, -57.384346, -75.4025345, -8.09004307, 21.5168056, 26.1001949, -11.5577393}
(gdb) stepi
26      add r1d, 1 ;sumo 1 al contador de bloques
(gdb) p/f $ymm1.v8_float
$7 = {-39.5025101, -12.5138882, 97.5386276, 98.0559158, -19.9952335, 88.0046082, 38.1431694, 52.1846088}
(gdb) p/f $ymm0.v8_float
$8 = {18.1453743, -2.82135296, 40.1542816, 22.6533813, -28.0852776, 109.521416, 64.2433624, 48.6268616}
(gdb) continue
Continuing.
N          = 16
Suma       = 264.438049
Media      = 16.527378
Varianza   = 2658.562256
StdDev     = 51.561249
Mínimo     = -75.402534
Máximo     = 98.055916
Tiempo promedio del kernel (1 rep.): 107827.176050 ms
[Inferior 1 (process 15595) exited normally]
(gdb)

```

Figura 17: Inspección de datos flotantes en registros YMM y finalización.

En la Figura 17 se inspeccionan los valores contenidos en los registros vectorial en formato de coma flotante de 32 bits mediante `p/f $ymm1.v8_float` y `$ymm0.v8_float` para ambos ciclos de ejecución del programa. Se aprecia cómo los datos del arreglo se van acumulando en `ymm0` a medida que se avanza paso a paso con `stepi`. Finalmente, la ejecución concluye exitosamente desplegando en consola las estadísticas del vector ($N = 16$, Suma, Media, Varianza, Desviación Estándar, Mínimo y Máximo).

VII-B. Sesión 2: Inspección de `normalize_array` y Verificación en Memoria

```

(gdb) set args data/input.dat data/output.dat 1
(gdb) break main
Breakpoint 1 at 0x1130: file src/driver.c, line 126.
(gdb) run
Starting program: /home/alejandro-henao/Proyecto_1_Arqui/proyecto_vectorial/proyecto_vectorial/bin/norm_vector data/input.dat data/output.dat 1

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main (argc=4, argv=0x7fffffffd38) at src/driver.c:126
126  int main(int argc, char **argv) {
(gdb) break normalize_array
Breakpoint 2 at 0x55555555c00: file asm/vector/stats_vector.asm, line 312.
(gdb) x/i 'normalize_array.end'
0x55555555cbe <normalize_array.end>:      vzeroupper
(gdb) b *0x55555555cbe
Breakpoint 3 at 0x55555555cbe: file asm/vector/stats_vector.asm, line 390.
(gdb)

```

Figura 18: Configuración de breakpoints para la función de normalización.

En la Figura 18 se inicia una segunda sesión de depuración enfocada en la función de normalización. Se coloca un punto de interrupción a la entrada de `normalize_array` (`break normalize_array`) y otro al final en la etiqueta `.end` justo en la instrucción `vzeroupper` (`b *0x55555555cbe`).

```

(gdb) continue
Continuing.

Breakpoint 2, normalize_array () at asm/vector/stats_vector.asm:312
312      cmp     edx, 0
(gdb) p/x $rsi
$1 = 0x555555559520
(gdb) set $output = $rsi
(gdb) delete 2
(gdb) continue
Continuing.

Breakpoint 3, normalize_array[end] () at asm/vector/stats_vector.asm:390
390      vzeroupper
(gdb) x/16fw $output
0x555555559520: 0.797507942   -0.132573262   -1.4334743    -1.78292644
0x555555559530: -0.477440327   0.0967670009   0.18565914    -0.544694304
0x555555559540: -1.08666658    -0.563222706   1.57116532    1.58119786
0x555555559550: -0.708334565   1.38625884     0.419225514   0.691550791
(gdb)

```

Figura 19: Inspección en memoria de la salida normalizada.

En la Figura 19 el flujo se detiene a la entrada de la función y se guarda la dirección del buffer de salida enviada en el registro \$rsi dentro de una variable de GDB (set \$output = \$rsi). Tras continuar hasta el breakpoint final (normalize_array[end]), se inspecciona el contenido en memoria del arreglo procesado usando x/16fw \$output. Se verifica que los 16 elementos de coma flotante fueron normalizados correctamente en memoria antes de limpiar el estado del procesador con vzeroupper.

VIII. CONCLUSIONES, LIMITACIONES Y TRABAJO FUTURO

Se logró la implementación y verificación adecuada de la versión escalar y vectorial AVX2 (registros YMM de 256 bits) capaces de realizar los cálculos de: suma, resta, media, varianza, desviación estándar, mínimo, máximo y normalización z-score sobre arreglos punto flotante, cumpliendo con la tolerancia relativa de 1×10^{-4} en todos los casos de prueba, incluyendo los casos borde y arreglos extremos.

La implementación escalar utilizando AVX2 demostró ganancias de rendimiento significativas cuando el conjunto de datos cabe en las cachés L1/L2, alcanzando un speedup máximo de 9,14x en $N = 2048$, cercano al límite teórico de 8x propio del procesador de 8 floats por instrucción.

Se confirmó que el speedup depende de la jerarquía de memoria, al superar los umbrales L1, L2 y L3, el speedup decae hasta $2,32x - 3,63x$, mostrando que el sistema pasa de estar limitado por cómputo a estar limitado por ancho de banda de memoria.

La gestión de remanentes (tail loop) mediante procesamiento escalar y alineación estricta de 32 bytes fueron indispensables para corrección numérica y prevenir fallos de segmentación, validando de esta manera las restricciones impuestas por la ABI System V y por el hardware AVX2.

VIII-A. Limitaciones

El speed up ideal de 8x nunca se alcanzó de forma sostenida y con las limitaciones de ancho de banda de memoria en arreglos grandes.

Para N muy pequeño ($N = 1$), la versión vectorial resultó ser más lenta que la escalar (157,5 ns/elemento vs 141,0 ns/elemento), debido al costo fijo de inicialización que tienen los registros YMM y la lógica de división para los bloques / residuo, lo que indica que la implementación vectorial no es ventajosa para cargas de trabajo triviales.

Las mediciones realizadas sin fijar núcleo presentaron altas dispersiones, debido a la migración entre núcleos y al escalado de frecuencia bajo el powersave, limitando de esta manera la precisión de comparaciones absolutas entre ejecuciones.

La estrategia de reducción horizontal implementada introduce un overhead adicional por cada llamada, ya que AVX2 carece de una instrucción nativa de reducción horizontal para 256 bits.

VIII-B. Trabajo futuro

Implementar un prefetching de forma explícita en la memoria para mitigar el efecto del ancho de banda.

Evaluar el rendimiento en las arquitecturas con soporte para Fused Multiply-Add, vfmadd231ps, esto para reducir el número de instrucciones en el cálculo de la varianza.

Repetir benchmarks fijando la frecuencia del CPU para aislar el efecto de la vectorización del ruido generado por el escalado dinámico de la frecuencia.

IX. ANEXO A: DIAGRAMAS DE ESTRUCTURA, TABLAS DE REGISTROS Y DE MEMORIA

El siguiente anexo detalla la estructura y funcionamiento del proyecto centrado en la implementación de kernels vectoriales SIMD/AVX y su comparación con el enfoque escalar. Se describe la interacción entre los módulos driver.c, libstats_scalar.asm y libstats_vector.asm, especificando el paso de parámetros y la preservación de registros según la ABI System V. A través de diagramas de flujo, se ilustra el control de bucles de cada kernel, desde la inicialización de acumuladores hasta la transición entre el bucle vectorial (8 elementos por iteración en registros YMM) y el procesamiento escalar para elementos remanentes. Esto resalta la diferencia estructural fundamental entre ambos enfoques. Finalmente, se integran tablas de asignación de registros

(YMM0, YMM1, RDI, RSI, etc.) y un diagrama de memoria. Este último detalla la alineación de arreglos a 32 bytes y contrasta los desplazamientos de puntero de 4 bytes en procesamiento escalar frente a los 32 bytes en procesamiento vectorial.

IX-A. Diagrama de bloques de la arquitectura de software

A continuación se muestra el diagrama de bloques de la arquitectura del software:

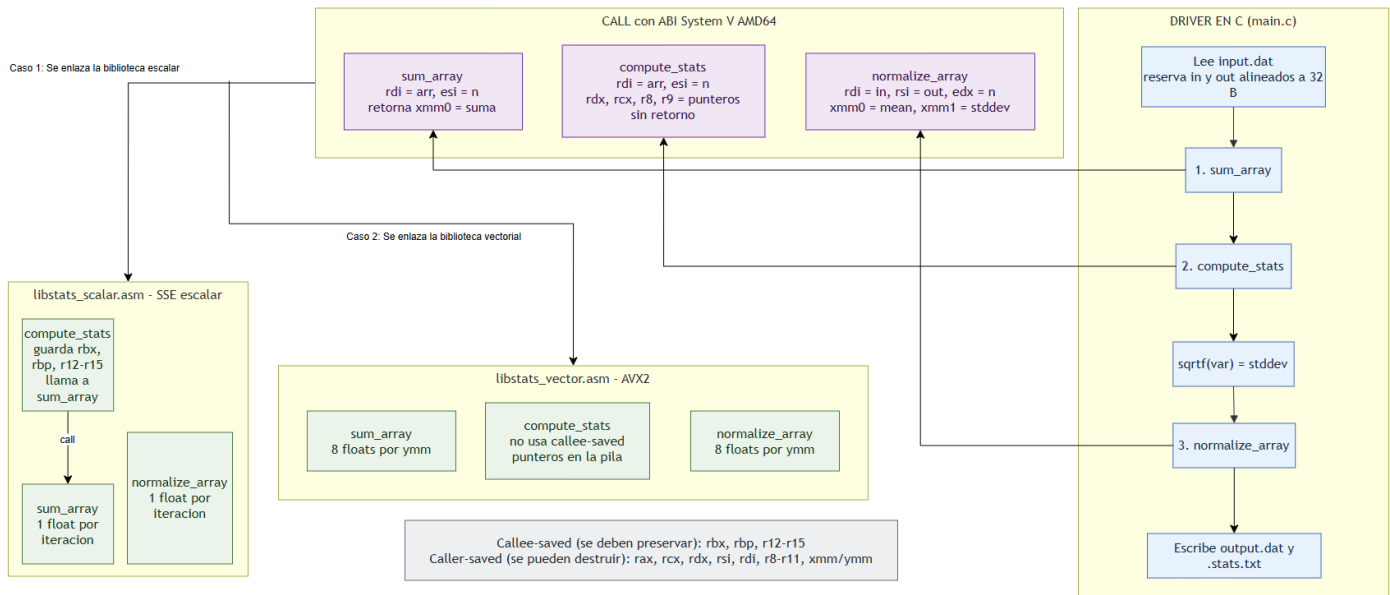


Figura 20: Diagrama de bloques de la arquitectura de software

El driver en C lee un archivo de extensión .dat (input.dat), reserva los arreglos alineados a 32 bytes y llama en orden a sum_array, compute_stats y normalize_array. Entre la segunda y la tercera calcula $\text{stddev} = \text{sqrtf}(\text{var})$. Cada llamada sigue la ABI System V: los enteros y punteros viajan en rdi, esi/rsi, rdx, rcx, r8 y r9, y los flotantes en xmm0 y xmm1. Solo sum_array devuelve un valor, en xmm0; compute_stats deja sus resultados en las direcciones que recibe por puntero, y normalize_array escribe en out. Las dos librerías definen los mismos tres símbolos, así que el driver no cambia: se enlaza una u otra. La escalar procesa un float por iteración, y su compute_stats guarda rbx, rbp y r12-r15 (callee-saved) con push/pop porque necesita conservar sus datos mientras hace call sum_array. La vectorial procesa 8 floats por registro ymm, no llama a ninguna otra función, no toca los callee-saved y guarda los punteros de salida en la pila. Por eso depende de que el driver entregue la memoria alineada a 32 bytes.

IX-B. Diagramas de flujo de ambos kernel

Por conveniencia y debido a la extensión de algunos de los diagramas. Todos los diagramas de flujo correspondientes a las diferentes funciones tanto del kernel escalar como del kernel vectorial se encuentran disponibles y pueden consultarse en el siguiente enlace: [Diagramas Proyecto 1: Kernel escalar y vectorial](#). Estos se encuentran organizados en seis pestañas independientes del documento, organizadas según el tipo de procesamiento y la función correspondiente. Se debe abrir el documento con Draw.io.

IX-C. Asignación de registros del Kernel Vectorial

A continuación se presentan unas tablas de resumen del contenido y función de los registros utilizados por el kernel vectorial por cada una de sus funciones. Dicha tabla se encuentra ordenada según la etiqueta correspondiente en el código de ensamblador.

Tabla IV: Asignación de Registros para sum_array vectorial

Etiqueta	Registro	Rol / Función Principal	Contenido / Valor Clave
sum_array	EAX	División	n antes de div / Cociente ($n/8$) después
	ESI	Parámetro de entrada	n (cantidad de elementos)
	EDX	División	Residuo ($n \bmod 8$)
	ECX	Divisor	Constante 8
	R8D	Contador	Bloques vectoriales procesados (inicia en 0)
	R9D	Contador	Elementos residuo procesados (inicia en 0)
	RDI	Puntero de lectura	Dirección del arreglo (arr)
	YMM0	Acumulador	Limpio (0,0)
	XMM1	Acumulador	Limpio (0,0)
.vector_sum	RDI	Puntero de lectura	Avanza +32 B por bloque
	YMM1	Registro de carga	Bloque actual (8 floats)
	YMM0	Acumulador	Suma parcial acumulada
	R8D	Contador	Incremento +1 bloque
	EAX	Límite del bucle	Total de bloques
.horizontal_sum	YMM0	Fuente de reducción	Acumulador vectorial completo
	XMM1	Acumulador escalar	Reducción parcial $\rightarrow$ Suma en xmm1[0]
	XMM2	Auxiliar de extracción	Mitad baja de YMM0
.remainder_sum	EDX	Límite del bucle	Cantidad de elementos en residuo
	RDI	Puntero de lectura	Avanza +4 B por float
	XMM1	Acumulador escalar	xmm1[0] += [rdi]
	R9D	Contador	Incremento +1 elemento
.ret_arr_sum	XMM1	Salida parcial	Suma total del arreglo
	XMM0	Retorno de función	Resultado final (float)

Tabla V: Asignación de Registros para compute_stats vectorial

Etiqueta	Registro	Rol / Función Principal	Contenido / Valor Clave
compute_stats	RDI / RSI / XMM0	Parámetros de entrada	Ptr. arr / Cantidad n / Suma total
	RDX / RCX / R8 / R9	Punteros de salida	Direcciones de &mean, &var, &min, &max
.stack_save	RSP / Otros	Preservación	Resguardo de punteros de salida y arr en pila
.mean	XMM1 / XMM0	Cálculo de media	Convierte n a float $\rightarrow$ Media ($\mu = sum/n$)
	RAX	Puntero de salida	Recupera &mean y escribe resultado
.set_up	YMM1 / EAX / EDX	Inicialización	Acumulador en 0,0 / Bloques ($n/8$) / Residuo ($n \bmod 8$)
.quad_vectorial	YMM2 / YMM4 YMM3 / YMM1	Carga / Broadcast Cálculo de varianza	Bloque actual (8 floats) / Copias de μ Diferencias $(x_i - \mu)$ / Acumula $(x_i - \mu)^2$
	RDI / R10D	Lectura / Contador	Avanza +32 B / Incrementa bloque
.horizontal_sum	YMM1 $\rightarrow$ XMM1	Reducción	Convierte suma cuadrática vectorial a escalar
.quad_remainder	XMM3 / XMM2 / XMM1	Residuos	Carga x_i / Calcula $(x_i - \mu)^2$ / Acumula escalar
.var_calc	XMM4 / XMM1 / RAX	Cálculo final	Convierte n a float $\rightarrow$ Varianza $\rightarrow$ Escribe &var
.min_setup	RDI / EAX / EDX	Reinicio	Restaura arr / Revisa bloques y residuos
.min_vectorial	YMM2 / YMM1 / R10D	Búsqueda mín. vec.	Carga bloque / Actualiza mínimos vec. / Incr. bloque
.horizontal_min	XMM1 / XMM2,3	Reducción mín.	Mínimo global de bloques en xmm1[0]
.min_remainder_*	XMM2 / XMM1	Residuos / Escalar	Carga x_i / Actualiza Mínimo final
.min_return	RAX	Puntero de salida	Recupera &min y escribe resultado
.max_setup	RDI / EAX / EDX	Reinicio	Restaura arr / Revisa bloques y residuos
.max_vectorial	YMM2 / YMM1 / R10D	Búsqueda máx. vec.	Carga bloque / Actualiza máximos vec. / Incr. bloque
.horizontal_max	XMM1 / XMM2,3	Reducción máx.	Máximo global de bloques en xmm1[0]
.max_remainder_*	XMM2 / XMM1	Residuos / Escalar	Carga x_i / Actualiza Máximo final
.max_return	RAX	Puntero de salida	Recupera &max y escribe resultado
.neqz ($n = 0$)	XMM0 / RAX	Caso borde	Asigna 0,0 a todas las salidas
.rsp_return	RSP	Control de pila	Restaura la posición del stack

Tabla VI: Asignación de Registros para normalize_array vectorial

Etiqueta	Registro	Rol / Función Principal	Contenido / Valor Clave
normalize_array	RDI / RSI / EDX	Entradas	Ptr. in / Ptr. out / Tamaño n
	XMM0 / XMM1	Parámetros estadísticos	Media (μ) / Desviación estándar (σ)
.set_up	EAX / EDX / R8D,R9D	Control de bucle	Bloques ($n/8$) / Residuo ($n \bmod 8$) / Contadores en 0
.stddev_eqz	XMM4 / XMM1	Caso especial ($\sigma = 0$)	Compara σ con 0,0 para evitar división por cero
.normalize_blocks	YMM2 / YMM3	Broadcast	Copias de μ (8) / Copias de σ (8)
	YMM5	Procesamiento vectorial	Carga $in[i] \rightarrow Calcula \frac{in[i]-\mu}{\sigma} \rightarrow$ Escribe out +32 B entrada / +32 B salida / Incr. bloque
.normalize_remainder	RDI / RSI / R8D	Avance de memoria	
	XMM5	Procesamiento escalar	Carga $x_i \rightarrow Calcula \frac{x_i-\mu}{\sigma} \rightarrow$ Escribe out +4 B entrada / +4 B salida / Incr. residuo
.block_loop_bc	RDI / RSI / R9D	Avance de memoria	
.block_loop_bc	YMM5	Copia directa ($\sigma = 0$)	Copia 8 floats de in a out sin modificar
.reminder_loop_bc	XMM5	Copia directa ($\sigma = 0$)	Copia 1 float de in a out sin modificar
.end	—	Salida	Ejecuta vzeroupper y retorna

IX-D. Asignación de registros del Kernel Escalar

A continuación una tabla resumen del contenido y función de los registros utilizados por el kernel escalar por cada una de sus funciones. Dicha tabla se encuentra ordenada según la etiqueta correspondiente en el código de ensamblador.

Tabla VII: Asignación de Registros para sum_array escalar

Etiqueta	Registro	Rol / Función Principal	Contenido / Valor Clave
sum_array	RDI	Parámetro de entrada	Puntero al arreglo (arr)
	ESI	Parámetro de entrada	Cantidad de elementos (n)
	EAX	Contador / Índice	Índice de iteración i (inicia en 0)
	XMM0	Acumulador / Retorno	Suma acumulada de elementos (0,0 al inicio)
.sum_loop	EAX	Control de bucle	Incremento +1 por elemento / Compara $i \geq n$
	XMM1	Registro de carga	Elemento actual arr[i]
	XMM0	Acumulador escalar	$xmm0 += xmm1$
.sum_done	XMM0	Retorno de función	Suma total del arreglo (float)

Tabla VIII: Asignación de Registros para compute_stats escalar

Etiqueta	Registro	Rol / Función Principal	Contenido / Valor Clave
compute_stats	RDI / ESI RDX / RCX / R8 / R9 RBX / EBP / R12–R15	Parámetros de entrada Punteros de salida Preservación	Puntero arr / Tamaño n Direcciones &mean, &var, &min, &max Copia segura de punteros y n (callee-saved)
.compute_stats_arreglo_vacio	XMM0	Caso borde ($n \leq 0$)	Escribe 0,0 en todas las direcciones de salida
Cálculo Media	RDI / ESI XMM0 / XMM4	Parámetros para llamada Cálculo de media	Copia de arr y n para call sum_array Suma total $\rightarrow$ Convierte n a float $\rightarrow$ Media ($\mu = sum/n$)
	R12 / XMM6	Almacenamiento	Escribe μ en *mean / Resguardo de μ en xmm6
.compute_stats_loop_principal	EAX XMM1 / XMM2	Contador / Índice Mínimo / Máximo	Índice de iteración i (inicia en 0) Mantenimiento de min y max acumulados
	XMM3	Registro de carga/aux	Elemento $x_i \rightarrow (x_i - \mu) \rightarrow (x_i - \mu)^2$
	XMM5	Acumulador varianza	Suma parcial cuadrática $\sum (x_i - \mu)^2$
.compute_stats_loop_final	R14 / R15	Punteros de salida	Escribe resultado final en *min y *max
	XMM5 / XMM4 / R13	Cálculo de varianza	Divide acumulado entre $n \rightarrow$ Escribe en *var
.compute_stats_retornar	RBX / EBP / R12–R15	final de función	Restaura registros callee-saved desde la pila

Tabla IX: Asignación de Registros para normalize_array escalar

Etiqueta	Registro	Rol / Función Principal	Contenido / Valor Clave
normalize_array	RDI / RSI / EDX	Entradas	Ptr. in / Ptr. out / Tamaño n
	XMM0 / XMM1	Parámetros estadísticos	Media (μ) / Desviación estándar (σ)
	XMM6 / XMM7	Resguardo de parámetros	Copia fija de μ y σ durante toda la función
Validación σ	XMM2 / XMM3	Filtro por valor abs.	Compara $ \sigma $ contra epsilon ($1,0 \times 10^{-6}$)
.normalize_array_loop	EAX XMM3	Contador / Índice Procesamiento escalar	Índice de iteración i (inicia en 0) Carga $in[i] \rightarrow$ Calcula $\frac{in[i] - \mu}{\sigma} \rightarrow$ Escribe out[i]
	RDI / RSI	Avance de memoria	Desplazamiento por índice +4 B ($rax \times 4$)
.normalize_array_loop_copia	EAX XMM3	Contador / Índice Copia directa	Bucle de respaldo si $\sigma \approx 0$ Lee $in[i]$ y lo escribe directamente en $out[i]$
.normalize_array_finalizar	—	Salida	Retorna de la función (ret)

IX-E. Diagramas de memoria de ambos kernel y alineación de memoria

A continuación se muestra el manejo de punteros para el procesamiento de los datos tanto para el kernel vectorial como escalar en el siguiente diagrama:

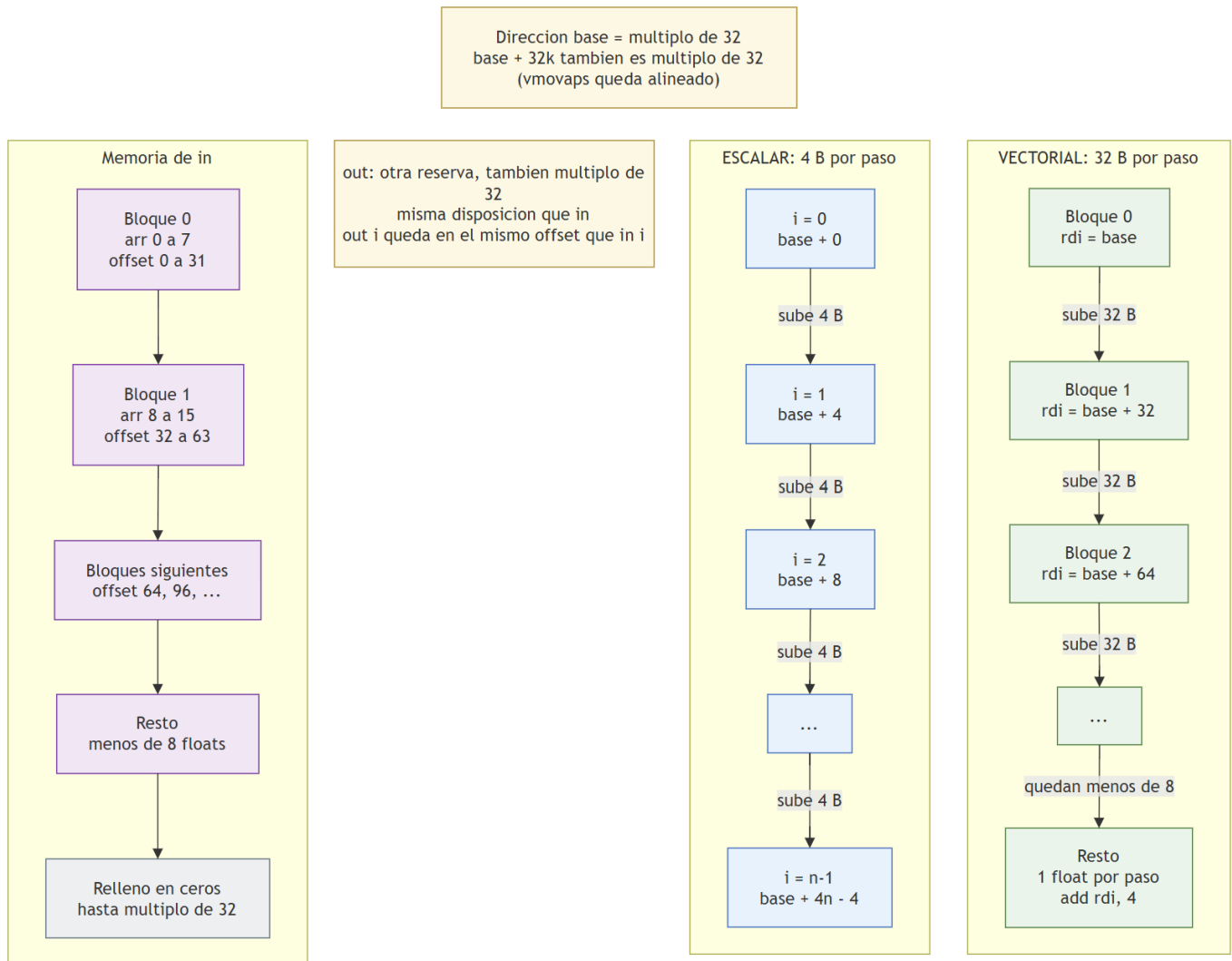


Figura 21: Diagrama de memoria para kernel escalar y vectorial

El diagrama muestra cómo se organiza en memoria el arreglo `in` y cómo lo recorre cada versión. Todo parte de la dirección `base`, que viene de `aligned_alloc(32, ...)` y por lo tanto es múltiplo de 32. Como el arreglo se divide en bloques de 32 bytes (8 floats de 4 bytes cada uno), el bloque 0 empieza en el offset 0, el bloque 1 en el 32, el siguiente en el 64, y así sucesivamente. Todas esas direcciones también son múltiplos de 32, y eso es lo que permite que `vmovaps` cargue cada bloque alineado. Después de los bloques completos queda un resto de menos de 8 floats, y al final hay un relleno en ceros hasta completar un múltiplo de 32, porque `aligned_alloc` exige ese tamaño. Las otras dos ramas muestran cómo se mueve el puntero: en la versión escalar la dirección de cada elemento es `base + 4i`, así que se avanza 4 bytes por paso hasta el último elemento. En la vectorial, `rdi` sube 32 bytes por bloque con `add rdi, 32`, y cuando quedan menos de 8 floats pasa a procesarlos de uno en uno con `add rdi, 4`. Por último, `out` es una reserva aparte, también alineada a 32 bytes y con la misma disposición, de modo que `out[i]` queda en el mismo offset que `in[i]`.

X. ENLACE AL REPOSITORIO DE GITHUB

https://github.com/Ahenao16/Proyecto_1_Arqui.git

REFERENCIAS

- [1] A. Henao y M. P. Camacho, Diagramas Proyecto 1: Kernel escalar y vectorial. Draw.io, 2026. [En línea]. Disponible en: <https://drive.google.com/file/d/1NXIoybkjvsID6XIHq0hD54FvY7OQUrl/view?usp=sharing>
- [2] *Intel® 64 and IA-32 Architectures Software Developer's Manual*. (n.d.). 2.
- [3] *Manuals for Intel® 64 and IA-32 architectures*. (n.d.). Intel. Retrieved September 18, 2026, from <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>
- [4] John Hennessy y David A. Patterson, *Computer Architecture: A Quantitative Approach*, 6^{ta} ed., Elsevier/Morgan Kaufmann. [En línea]. Disponible en [https://acs.pub.ro/~cpop/SMPA/Computer%20Architecture,%20Sixth%20Edition_%20A%20Quantitative%20Approach%20\(%20PDFDrive%20\).pdf](https://acs.pub.ro/~cpop/SMPA/Computer%20Architecture,%20Sixth%20Edition_%20A%20Quantitative%20Approach%20(%20PDFDrive%20).pdf)
- [5] Patterson, D. A., & Hennessy, J. L. (2015). *Computer organization and design: The hardware/software interface* (Interactive ed.). Morgan Kaufmann. [En línea]. Disponible en <https://books.google.com.bo/books?id=U6lpXmu7OngC&printsec=copyright&hl=es#v=onepage&q&f=false>